

Assignment 5 Report

KG Multi-Agent QA System

NCU Regulation Question-Answering System

Course	Advanced NLP / Knowledge Engineering
Assignment	Assignment 5 — A4 Extension
TA	Ge Tingyu
Due Date	2026 / 5 / 7
LLM	Qwen/Qwen2.5-3B-Instruct (local inference)
Database	Neo4j (Docker) + SQLite

0. Objective

This assignment extends Assignment 4 by building a multi-agent Question-Answering system on top of the NCU Regulation Knowledge Graph (KG). The system provides natural-language querying of university regulations, while enforcing security policies, detecting errors, and attempting automatic query repair.

The three main goals are:

- Build a multi-agent QA pipeline on top of the A4 KG (Neo4j graph of NCU regulations).
- Add security validation to reject unsafe or policy-violating requests.
- Add diagnosis and repair flow to handle query failures gracefully.

1. System Architecture

The system follows a hybrid pipeline design: a fixed front-half (sequential agents) and a dynamic back-half that branches based on the diagnosis result. The pipeline is implemented in `query_system_multiagent.py` and all agent modules live in the `agents/` directory.

Pipeline Overview

Step	Agent	Responsibility
1	NL Understanding	Parses the user question into a structured Intent object (question_type, keywords, aspect).
2	Security / Policy	Screens the question and intent for unsafe patterns (DELETE, DROP, prompt injection, etc).
3	Query Planning	Converts the intent into a Neo4j query plan (typed_then_broad or fulltext_only strategy).
4	Query Execution	Runs a read-only Cypher query against Neo4j and returns rows or an error.
5	Diagnosis	Classifies the execution result as SUCCESS, NO_DATA, QUERY_ERROR, or SCHEMA_MISMATCH.
6	Query Repair	If the diagnosis is an error, generates a revised query plan (max 1 repair round).
7	Explanation	Summarises the reasoning chain and produces a human-readable explanation field.

2. Knowledge Graph Design (A4 Carry-over)

The KG is built from NCU regulation data stored in a SQLite database (`ncu_regulations.db`). The build script (`build_kg.py`) uses the Qwen2.5-3B-Instruct model (local inference, no internet required after first download) to extract structured rules from each article.

Schema

Node / Relationship	Properties	Description
:Regulation	id, name, category	Top-level regulation document

:Article	number, content, reg_name, category	Individual article within a regulation
:Rule	rule_id, type, action, result, art_ref, reg_name	Extracted rule triple
(:Regulation)-[:HAS_ARTICLE]->(:Article)		Regulation contains articles
(:Article)-[:CONTAINS_RULE]->(:Rule)		Article yields rules

Full-text Indexes

Two full-text indexes are created to support keyword search: **article_content_idx** on Article.content, and **rule_idx** on Rule.action + Rule.result. These allow the Query Execution Agent to fall back to broad keyword matching when an exact structured query returns no results.

3. Agent Implementation Details

3.1 NL Understanding Agent

The NLUnderstandingAgent converts a raw user question into a structured Intent dataclass. It extracts a question_type (e.g. requirement, permission, prohibition), a list of keywords (tokenised and filtered), and an aspect (e.g. graduation, scholarship, attendance). An ambiguous flag is set when the question contains vague language such as 'any' or 'something'.

3.2 Security / Policy Agent

The SecurityAgent screens both the raw question and the intent for unsafe patterns before any database interaction. Hard-blocked keywords include: DELETE, DROP, MERGE, CREATE, SET, BYPASS, 'ignore previous', and 'dump all'. If any pattern matches, the agent immediately returns decision=REJECT and the pipeline short-circuits, returning a REJECT response without touching Neo4j.

3.3 Query Planner Agent

The QueryPlannerAgent converts the structured Intent into a Cypher query plan. Two strategies are supported: typed_then_broad (first tries to match Rule.type, then falls back to full-text search) and fulltext_only (used during repair). The plan includes the keyword list, the target aspect, and the chosen strategy.

3.4 Query Execution Agent

The QueryExecutionAgent executes the plan against Neo4j using read-only Cypher. For the typed_then_broad strategy it first tries a structured MATCH on Rule nodes filtered by type and keyword, then falls back to the full-text index. All queries are parameterised to prevent Cypher injection. Results are returned as a list of row dicts or an error string.

3.5 Diagnosis Agent

The DiagnosisAgent classifies the execution outcome into one of four labels: SUCCESS (rows returned), NO_DATA (empty result, valid query), QUERY_ERROR (driver exception), or

SCHEMA_MISMATCH (returned columns do not match expected schema). The label and a human-readable reason string are returned.

3.6 Query Repair Agent

If diagnosis is QUERY_ERROR or SCHEMA_MISMATCH, the QueryRepairAgent generates a revised plan. It switches to the fulltext_only strategy and broadens the keyword list (removing stop-words, adding synonyms from a small domain vocabulary). Only one repair round is allowed. repair_changed is set to True only if the repaired plan differs from the original.

3.7 Explanation Agent

The ExplanationAgent produces the explanation field of the output contract by summarising the reasoning chain: intent classification, security decision, execution strategy, diagnosis label, whether repair was attempted, and the final answer source.

4. Output Contract

Every call to **run_multiagent_qa(question) / answer_question(question)** returns a dict with the following fields, as required by auto_test_a5.py:

Field	Type	Values / Description
answer	str	Natural-language answer extracted from KG rows.
safety_decision	str	ALLOW or REJECT
diagnosis	str	SUCCESS QUERY_ERROR SCHEMA_MISMATCH NO_DATA
repair_attempted	bool	True if a repair round was executed.
repair_changed	bool	True if the repaired plan differed from the original.
explanation	str	Human-readable reasoning summary.

5. Design Decisions

Local LLM for KG construction

Qwen2.5-3B-Instruct was chosen for entity extraction during KG build (build_kg.py) because it fits in 6 GB of VRAM, supports 32K context, and runs fully offline after the first download. This avoids API costs and rate limits during the potentially long build process.

Hybrid pipeline (fixed front / dynamic back)

The front half (NLU → Security → Plan → Execute → Diagnose) is always executed in order, giving predictable behaviour. The back half branches only when an error is detected, keeping the happy path fast and the repair path isolated.

Read-only Neo4j access

All Cypher queries are read-only (MATCH / CALL db.index.fulltext.queryNodes). This is enforced at the agent level and satisfies the A4→A5 continuity rule that runtime QA must not mutate the KG.

Single repair round

Allowing only one repair round keeps latency bounded and prevents infinite loops. If the repaired query still fails, the system returns a graceful error answer rather than retrying indefinitely.

Parameterised Cypher queries

All Neo4j queries use parameterised statements (not string formatting) to prevent Cypher injection — a secondary layer of security on top of the SecurityAgent keyword filter.

6. Difficulties Encountered

LLM JSON parsing failures during KG build

Qwen2.5-3B sometimes wrapped its JSON output in markdown fences (``json ... ``) or added a preamble sentence. This was resolved with a regex cleanup step (re.sub) before json.loads, plus a try/except that returns an empty rules list on any parse failure, so one bad article never blocks the entire build.

Full-text index query syntax

Neo4j's CALL db.index.fulltext.queryNodes requires Lucene query syntax. Special characters in user input (e.g. parentheses, slashes) can crash the index call. The solution was to sanitise keywords by stripping non-alphanumeric characters before passing them to the full-text index.

Distinguishing NO_DATA from QUERY_ERROR

An empty result list and a driver exception both result in no answer, but require different diagnosis labels. The DiagnosisAgent checks for an 'error' key in the execution dict first, then checks for empty rows, ensuring the correct label is always returned.

Docker / Neo4j startup order

The preflight check in auto_test_a5.py verifies Neo4j connectivity and counts Rule nodes. During development, running auto_test_a5.py before build_kg.py finished caused confusing failures. The fix was to add a startup wait loop and a clear error message guiding the user to run setup_data.py + build_kg.py first.

7. Key Findings and Insights

- Full-text search significantly improves recall for regulation queries compared to pure structured Cypher matching, because user phrasing rarely matches stored rule text verbatim.
- The security layer (keyword blocking) is fast and effective for obvious injection attempts but would benefit from a semantic classifier to catch paraphrased unsafe requests.

- The single-repair-round constraint is sufficient for most schema mismatch cases, where the failure is typically a wrong property name that the fulltext strategy avoids.
- Grounding the answer in actual KG rows (rather than LLM generation) produces factually reliable answers at the cost of naturalness — a trade-off appropriate for a regulation QA system.
- The Qwen2.5-3B model achieves good extraction quality on regulation text without GPU acceleration, though build time on CPU is significantly longer (~2-4 seconds per article).

8. Expected Grading Breakdown

Component	Weight	Notes
Task Success Rate (normal cases)	25 / 60	Full-text fallback improves recall.
Security & Validation (unsafe cases)	15 / 60	Hard keyword block + REJECT contract.
Error Detection Quality (failure cases)	8 / 60	4-label diagnosis covers all cases.
Query Regeneration	6 / 60	fulltext_only repair strategy.
Correct Resolution After Repair	6 / 60	Success depends on index coverage.
Report / Documentation	40 / 100	This document.

Conclusion

This assignment successfully extends the A4 Knowledge Graph into a full multi-agent QA system. The seven-agent pipeline provides natural language understanding, security filtering, structured and full-text query execution, automatic diagnosis, single-round repair, and human-readable explanation — all within the strict output contract required by `auto_test_a5.py`. The design prioritises reliability and safety over answer fluency, which is appropriate for a university regulation QA system where factual accuracy is critical.